# UNIDAD 3 – Semana 12

Asignatura: Desarrollo de Aplicaciones Web

**NRC:** **Fecha:**

**Estudiante:**

**Docente:**

---

El Hospital San Gabriel administra sus medicamentos, insumos médicos y materiales hospitalarios mediante hojas de cálculo y registros manuales. Esta situación ha generado dificultades para mantener actualizadas las existencias, controlar las entradas y salidas de productos, consultar el historial de movimientos y detectar medicamentos e insumos próximos a agotarse.

Ante esta problemática, la dirección del hospital ha solicitado desarrollar un sistema web que centralice la información del inventario y permita gestionar de manera organizada los productos almacenados y sus movimientos.

Para resolver el caso, deberán desarrollar una aplicación web utilizando Angular 22, Signal Forms, FastAPI y SQL Server. El backend deberá implementar una arquitectura por capas y utilizar procedimientos almacenados para las operaciones con la base de datos. Asimismo, el desarrollo deberá gestionarse mediante Git, ramas y Pull Requests.

**Requerimientos funcionales:**

**RF01. Gestión de productos hospitalarios**

El sistema deberá permitir:

- Registrar productos con código, nombre, descripción, categoría, unidad de medida, precio unitario, stock mínimo y fecha de vencimiento cuando corresponda.
- Listar y consultar los productos registrados.
- Actualizar la información de los productos.
- Desactivar productos sin eliminar su historial de movimientos.
- Buscar productos por código o nombre y filtrar por categoría.
- Identificar medicamentos e insumos próximos a vencer.

**RF02. Gestión de categorías**

El sistema deberá permitir:

- Registrar categorías de productos, como medicamentos, material médico, material de laboratorio y productos de limpieza hospitalaria.
- Listar las categorías disponibles.
- Asociar cada producto con una categoría.

**RF03. Registro de entradas**

El sistema deberá permitir:

- Registrar entradas indicando producto, cantidad, fecha, motivo, número de lote y fecha de vencimiento cuando corresponda.
- Registrar el ingreso de productos por compras, donaciones o devoluciones.
- Actualizar automáticamente las existencias.
- Registrar el movimiento correspondiente en el historial.

### RF04. Registro de salidas

El sistema deberá permitir:

- Registrar salidas indicando producto, cantidad, fecha, motivo y área hospitalaria de destino.
- Registrar salidas destinadas a servicios como emergencia, hospitalización, consultorios externos y laboratorio.
- Impedir que se retire una cantidad superior al stock disponible.
- Actualizar las existencias y registrar el movimiento correspondiente.
- Identificar el lote del producto retirado cuando corresponda.

### RF05. Historial de movimientos

El sistema deberá permitir:

- Consultar los movimientos del inventario.
- Filtrar los movimientos por producto, tipo de movimiento, área hospitalaria y rango de fechas.
- Mostrar el tipo de movimiento, cantidad, fecha, motivo, stock anterior y stock posterior.
- Consultar el lote relacionado con cada movimiento cuando corresponda.

### RF06. Consultas e indicadores

El sistema deberá mostrar:

- Cantidad de productos activos.
- Productos cuyo stock sea igual o inferior al stock mínimo.

- Productos próximos a vencer y productos vencidos.
- Valor referencial del inventario.
- Movimientos registrados dentro de un rango de fechas.
- Productos con mayores salidas durante un periodo determinado.

**Reglas de negocio**

La implementación deberá respetar las siguientes reglas:

1. El código de cada producto será único.
2. Los precios deberán ser mayores que cero.
3. Las cantidades de entrada y salida serán números enteros mayores que cero.
4. No se permitirán salidas que generen stock negativo.
5. Cada entrada o salida válida deberá generar un registro histórico.
6. La actualización del stock y el registro del movimiento deberán realizarse dentro de una misma transacción.
7. Los productos con movimientos registrados no podrán eliminarse físicamente; deberán desactivarse.
8. Los campos obligatorios y los datos ingresados deberán validarse.
9. Los productos vencidos no podrán registrarse como salidas para uso hospitalario.

10. Cuando un producto se gestione por lotes, deberá conservarse la trazabilidad de sus entradas, vencimientos y salidas.
11. Las salidas deberán identificar el área hospitalaria de destino.
12. Los indicadores y consultas deberán reflejar los datos actualizados de la base de datos.

**Requerimientos técnicos obligatorios**

**Frontend: Angular 22 y Signal Forms**

Deberán desarrollar una interfaz responsive que incluya:

- Inicio de sesión de usuarios.
- Panel principal con indicadores del inventario.
- Gestión de productos hospitalarios.
- Gestión de categorías.
- Registro de entradas y salidas.
- Historial de movimientos.
- Consulta de productos con stock bajo.
- Consulta de productos próximos a vencer y vencidos.

La implementación deberá cumplir las siguientes condiciones:

- Utilizar componentes y servicios de Angular.
- Implementar formularios mediante Signal Forms, incluyendo validaciones y mensajes de error.
- Consumir los endpoints de FastAPI mediante solicitudes HTTP.

- Obtener dinámicamente las categorías y demás datos necesarios desde el backend.
- Actualizar los listados y la información presentada después de registrar o modificar datos.
- Mantener una interfaz adaptable a computadoras, tabletas y dispositivos móviles.

**Backend: FastAPI y arquitectura por capas**

El backend deberá organizarse según la estructura vista en clase.

Los endpoints no deberán contener consultas SQL directas ni asumir responsabilidades propias de las capas Services o Repositories. El backend deberá gestionar los errores y devolver respuestas HTTP apropiadas.

**Base de datos: SQL Server y procedimientos almacenados**

Deberán diseñar e implementar una base de datos. El modelo deberá incluir claves primarias, claves foráneas, restricciones de integridad y tipos de datos apropiados.

Se deberán implementar procedimientos almacenados. Estos podrán organizarse de acuerdo con el diseño propuesto, siempre que se cubran todas las operaciones solicitadas.

Condiciones obligatorias:

- Todas las operaciones de persistencia deberán ejecutarse mediante procedimientos almacenados.
- Los procedimientos de entradas y salidas deberán utilizar transacciones para garantizar la consistencia de las existencias y del historial

- Las credenciales de conexión deberán mantenerse fuera del código fuente público.
- La gestión de lotes y vencimientos deberá permitir identificar los productos que no pueden utilizarse por estar vencidos.
- Todas las operaciones de persistencia deberán ejecutarse mediante procedimientos almacenados.
- Los procedimientos de entradas y salidas deberán utilizar transacciones para garantizar la consistencia del stock y del historial.
- Los cambios registrados en SQL Server deberán visualizarse al consultar nuevamente la información desde el frontend.
- Las credenciales de conexión deberán mantenerse fuera del código fuente público.

**Gestión del código fuente: Git, ramas y Pull Requests**

El desarrollo deberá gestionarse mediante un repositorio Git que evidencie el trabajo realizado.

Se deberá cumplir con las siguientes condiciones:

- Crear ramas para desarrollar funcionalidades.
- Registrar commits frecuentes y descriptivos.
- Crear Pull Requests para integrar los cambios desarrollados en las ramas.
- Mantener un historial de cambios organizado y comprensible.
- Incluir un archivo README.md con la descripción del proyecto y las instrucciones de instalación y ejecución.

La organización del repositorio deberá permitir identificar los cambios realizados y la forma en que se integraron las funcionalidades.

**Nota:**

1. Durante la sustentación se realizarán preguntas individuales sobre el código desarrollado, la arquitectura del sistema, los procedimientos almacenados, las reglas de negocio y las decisiones técnicas adoptadas. Si el estudiante no logra explicar partes sustanciales de su implementación, su calificación podrá verse afectada en los criterios correspondientes de la rúbrica, independientemente del funcionamiento aparente de la aplicación.
2. En caso de no sustentar el proyecto, la calificación será de cero (0).

# Rúbrica de evaluación

| Criterios | Excelente (4) | Bueno (3) | Regular (2) | Deficiente (0) |
|---|---|---|---|---|
| Funcionalidades y reglas de negocio | Implementa la gestión de productos y categorías, entradas, salidas, historial, consultas e indicadores. Aplica las validaciones, impide el stock negativo y conserva los movimientos históricos. | Implementa las principales funcionalidades y reglas de negocio, con omisiones o errores menores. | Implementa parcialmente las funcionalidades y presenta errores en validaciones, movimientos o consultas. | No implementa las funcionalidades esenciales o estas no funcionan correctamente. |
| Frontend | Implementa componentes, servicios e interfaces de Angular. Utiliza Signal Forms con validaciones y mensajes de error. Consume la API correctamente, obtiene dinámicamente datos como las categorías, presenta una interfaz visualmente atractiva y responsive. | Utiliza los recursos requeridos de Angular y Signal Forms, con errores menores de validación, integración o presentación. | Presenta formularios incompletos, uso parcial de los recursos requeridos o problemas de integración. Algunos datos están codificados directamente en el frontend. | No demuestra una implementación funcional con Angular 22 y Signal Forms, o la interfaz no permite realizar las operaciones esenciales. |
| Backend | Implementa correctamente las capas APIs, Services, Repositories y Schemas, con responsabilidades diferenciadas. Valida datos, gestiona errores y utiliza procedimientos almacenados para las operaciones de base de datos. La API devuelve información actualizada. | Implementa las cuatro capas con una separación generalmente adecuada y presenta deficiencias menores en validaciones, manejo de errores o integración. | Implementa parcialmente las capas, concentra responsabilidades en algunos componentes o presenta problemas de integración y actualización de datos. | No implementa la arquitectura solicitada o ejecuta consultas SQL directas en lugar de utilizar procedimientos almacenados. |

| | | | | |
|---|---|---|---|---|
| Base de datos | Diseña e implementa una base de datos con claves, relaciones, restricciones y tipos apropiados. Implementa los procedimientos almacenados para las operaciones e informes. Garantiza la consistencia de entradas y salidas mediante transacciones. Los cambios realizados en la base de datos se reflejan en la aplicación al consultar nuevamente los datos, sin modificar el código fuente. | Implementa el modelo y los procedimientos almacenados principales. Las transacciones y la actualización de datos funcionan, aunque existen deficiencias menores. | Presenta procedimientos incompletos, problemas de integridad o transacciones, o algunos datos no se actualizan correctamente en la aplicación. | No implementa adecuadamente la base de datos, omite los procedimientos almacenados requeridos o no garantiza la consistencia de las existencias. |
| Gestión de versiones | Mantiene un historial de commits frecuentes y descriptivos, utiliza ramas para desarrollar funcionalidades y crea Pull Requests para integrar cambios de manera organizada. | Utiliza ramas, commits y Pull Requests, con deficiencias menores en su organización o descripción. | Utiliza Git de forma limitada, presenta pocos commits descriptivos o evidencia dificultades en el uso de ramas y Pull Requests. | No demuestra el uso del flujo de trabajo solicitado con Git, ramas y Pull Requests. |